From Base Model to Frontier Reasoner

Multi-Stage Pipelines, Test-Time Compute, Distillation, and the Agentic Frontier

1

Learning Objectives

- Trace the **complete multi-stage post-training pipeline**: SFT → reasoning RL → rejection sampling / distillation → general alignment
- Explain the **DeepSeek-R1-Zero** phenomenon: pure RL on a base model produces emergent chain-of-thought, self-verification, and backtracking
- Understand **test-time compute scaling**: best-of-N, sequential reasoning, tree search + PRM, budget forcing — trading inference compute for accuracy
- Explain **distillation** as a stabilization and efficiency tool: rejection sampling, on-policy distillation, and specialist distillation
- Model agentic RL as **multi-turn interaction with environments**: tool use, reward stacks, and the actor–verifier–learner architecture
- Identify the **infrastructure bottleneck** — rollout generation — and how async architectures solve it
- Have a complete picture of how frontier reasoning models are built

2

From Last Time

- We have **optimization algorithms**: PPO, GRPO, DAPO — all variations of policy gradient + baseline + trust region
- We have **reward signals**: learned reward models (Bradley-Terry, GenRM), DPO (implicit), verifiable rewards (exact), process reward models (per-step)
- We know that **reward hacking** is the fundamental failure mode, and mitigations include KL penalties, RM ensembles, and mixing verified rewards
- **This lecture**: how to compose all these pieces into the actual training pipelines, inference strategies, and agentic systems that produce frontier models

3

Part 1: The Multi-Stage Pipeline

4

Why Not Just Run RL Once?

- Frontier models must be good at **many things**: reasoning, instruction following, safety, tool use, conversation, creativity
- Different capabilities require **different reward signals**: math needs verifiable rewards, helpfulness needs preferences, safety needs refusal criteria
- Training on all objectives at once is unstable — reward signals **conflict** (brevity vs. thoroughness, safety vs. helpfulness)
- **Solution**: decompose post-training into **sequential stages**, each targeting specific capabilities with matched reward signals
- Each stage starts from the previous stage's best checkpoint — **progressive refinement**, not monolithic optimization

5

The Canonical Multi-Stage Pipeline

```
INPUT: Pre-trained base model M_0

┌─ STAGE 1: SFT ──────────────────────────────────────────┐
│ Data: curated (prompt, response) pairs                  │
│   - instruction following, conversation                 │
│   - chain-of-thought with <think>...</think> format     │
│   - tool use, code generation                           │
│ Method: standard cross-entropy fine-tuning              │
│ Scale: 10K–100K examples, 1–3 epochs                    │
│ Output: M_1 (SFT model — knows the FORMAT of reasoning) │
└─────────────────────────────────────────────────────────┘
          ↓
┌─ STAGE 2: REASONING RL ─────────────────────────────────┐
│ Data: math/code/logic problems with verifiable answers  │
│ Method: GRPO/DAPO + verifiable rewards                  │
│   For each problem:                                     │
│     Sample G=16–64 solutions from current policy        │
│     Verify each: correct → r=1, incorrect → r=0         │
│     Group-normalize: Â_i = (r_i - μ) / σ                │
│     PPO-clip update on token-level ratios               │
│ Scale: 100K+ problems × 1000s of RL steps               │
│ Output: M_2 (reasoning model — knows HOW to reason)     │
└─────────────────────────────────────────────────────────┘
          ↓
┌─ STAGE 3: REJECTION SAMPLING + DISTILLATION ────────────┐
│ Use M_2 to generate N solutions per problem             │
│ Keep only verified-correct solutions (optionally top-k) │
│ Fine-tune M_1 (or M_2) on this curated dataset          │
│ Distills RL gains into clean supervised checkpoint      │
│ Output: M_3 (distilled reasoning model — more stable)   │
└─────────────────────────────────────────────────────────┘
          ↓
┌─ STAGE 4: GENERAL ALIGNMENT ────────────────────────────┐
│ Data: preference pairs (from LLM-as-judge or humans)    │
│ Method: DPO or PPO + reward model                       │
│ Objectives: helpfulness, safety, instruction following  │
│ KL penalty anchored to M_3 to preserve reasoning        │
│ Output: M_4 (final aligned model)                       │
└─────────────────────────────────────────────────────────┘

OUTPUT: M_4 — deployed model
```

Why This Ordering Matters

- **SFT before RL**: the model needs the *format* of reasoning (think step-by-step, write code blocks, use tools) before RL can improve the *quality*. Without SFT, RL has no structure to optimize over.
- **Reasoning RL before general alignment**: reasoning is **harder** to learn — requires exploring many solution paths. General alignment (be polite, be safe) is easier and can partially undo reasoning gains if done first.
- **Distillation between RL and alignment**: distills the noisy RL policy into a clean supervised checkpoint — more stable starting point for alignment.
- **General alignment last**: fine adjustments to tone, safety, instruction following. KL penalty to the reasoning model prevents forgetting.
- **Catastrophic forgetting** is the constant enemy: mitigations include KL penalties, low learning rates, short fine-tuning, replay of earlier-stage data.
- **Note**: DeepSeek-V3.2 merges reasoning, agentic tasks, and alignment into a **single mixed RL stage** rather than sequential stages — an emerging alternative that avoids catastrophic forgetting between stages but requires careful reward balancing

Part 2: The DeepSeek-R1 Story

8

R1-Zero: Emergent Reasoning from Pure RL

- **DeepSeek-R1-Zero** (January 2025): apply GRPO directly to the **base model** (DeepSeek-V3-Base) — no SFT, no curated reasoning examples
- Reward: purely verifiable — math correctness + format compliance (answer in `` tags)
- After thousands of RL iterations, the model **spontaneously developed**:
- • **Chain-of-thought reasoning** — long thinking traces before answering
- • **Self-verification** — checking its own intermediate steps
- • **Backtracking** — recognizing errors and trying alternative approaches
- • **Increasing thinking time** — generating more tokens on harder problems
- None of these behaviors were in the training signal — the reward only checked the final answer. **The reasoning strategies emerged because they increased the probability of getting correct answers.**

9

R1-Zero → R1: From Scientific Discovery to Deployable Model

- R1-Zero had **real problems** despite impressive reasoning:
- • Unreadable outputs: mixing languages, poor formatting, repetitive text
- • No instruction following: it could reason but couldn't have a normal conversation
- • Instability: reward hacking on format rules, length explosion
- **DeepSeek-R1** fixed this with the full multi-stage pipeline:
- 1. Cold-start SFT: thousands of curated long-CoT examples for readable reasoning format
- 2. Reasoning RL: GRPO + verifiable rewards (same as R1-Zero, better starting point)
- 3. Rejection sampling: generate many solutions, keep the best, fine-tune into clean data
- 4. General alignment RL: helpfulness, safety, instruction following via reward model
- **Key lesson**: RL discovers capabilities (reasoning emerges), SFT/distillation makes them usable. This pattern — **RL for capability, distillation for stability** — is now the standard at every frontier lab.

10

Part 3: Distillation as a Core Pipeline Component

11

Why Distillation Matters in the RL Pipeline

- RL produces powerful but **noisy** policies — the policy explores broadly, some behaviors are undesirable
- **Rejection sampling distillation**: generate many solutions with the RL model, keep only verified-correct ones, SFT on the curated set. Cleans up the policy.
- **Specialist distillation** (DeepSeek-V3.2): train separate specialist models per domain (math, code, agentic, general), then distill all specialists into a single unified model. Avoids multi-stage catastrophic forgetting.
- **On-policy distillation (OPD)**: the student generates its own trajectories, the teacher provides dense token-level supervision on those trajectories. Combines the on-policy benefit of RL with the dense supervision of SFT.
- OPD can be 10–100× cheaper than RL for comparable reasoning performance — an emerging alternative to full RL for smaller teams
- **R1 distillation results**: DeepSeek distilled R1 (671B) into smaller models (1.5B–70B). The distilled models significantly outperformed RL-trained models of the same size — **distillation from a strong teacher often beats RL from scratch.**

12

Distillation vs. RL: Complementary, Not Competing

- **RL** can discover new capabilities that no training data contains (R1-Zero proved this). It pushes the frontier.
- **Distillation** cannot exceed the teacher — it transfers existing capabilities efficiently. It broadens the frontier.
- The frontier recipe is: **RL creates the best model → distillation makes it accessible/efficient**
- RL shapes behavior but doesn't teach new knowledge well. SFT/distillation injects knowledge but doesn't shape behavior well.
- On-policy distillation sits at the intersection: dense supervision on the student's own states, combining benefits of both
- In practice, production pipelines alternate between RL and distillation stages multiple times

13

Part 4: Test-Time Compute Scaling

14

The Core Idea: Think Longer on Harder Problems

- Traditional scaling: make the model **bigger** (more parameters). Expensive, one-time cost at training.
- **Test-time compute scaling**: make the model **think longer** (more tokens at inference). Cost scales with difficulty.
- A 70B model generating 10,000 tokens of reasoning can outperform a 400B model generating 200 tokens — on math, code, and logic
- This is the insight behind OpenAI o1/o3, DeepSeek-R1, and all 'reasoning models'
- **Key enabler**: the model must have learned (via RL) that **more thinking = better answers** — it must know *how* to use extra tokens productively
- Two dimensions: **parallel scaling** (generate multiple outputs, aggregate) and **sequential scaling** (generate longer chains, self-correct)

15

Four Methods for Test-Time Scaling

- **1. Best-of-N sampling**: generate N independent responses, score with verifier/RM, return the best. Simple, embarrassingly parallel. Pass@1 → pass@64 jumps from 40% to 95%+ on many problems.
- **2. Sequential reasoning / long CoT**: let the model generate a long chain-of-thought with self-correction. Generate → evaluate internally → backtrack → retry. This is what 'thinking' models do (o1, R1).
- **3. Tree search with PRM**: at each step, generate multiple candidates → score with process reward model → expand promising branches → prune bad ones. Systematic exploration of the reasoning space.
- **4. Budget forcing**: explicitly control thinking tokens. Inject `` to start, force `` after B tokens. Route easy problems to short budgets, hard problems to long budgets.
- No single strategy universally dominates — the optimal choice depends on problem difficulty, model type, and compute budget

16

Test-Time Compute Methods Compared

Method	How It Works	Scaling Type	Strength	Weakness
Best-of-N	Generate N solutions, pick best	Parallel	Simple, fully parallelizable	No info sharing between samples
Long CoT	Extended reasoning with self-correction	Sequential	Dynamic, adaptive to difficulty	Model must have learned self-correction via RL
Tree search + PRM	Branch-and-bound over reasoning steps	Hybrid	Systematic exploration	Needs good PRM, complex orchestration
Budget forcing	Control thinking budget per problem	Sequential	Efficient compute allocation	Needs difficulty estimation

Can be combined. All require RL-trained models that know how to use extra tokens productively.

17

Test-Time Scaling for Agents

- Test-time scaling extends beyond single-turn reasoning to multi-turn agentic tasks
- For agents: parallel sampling means running N independent trajectories; sequential revision means the agent can retry tool calls or revise plans
- DeepSeek-V3.2 uses **context management** for test-time scaling in agentic tasks: when token usage exceeds 80% of the context window, it summarizes or discards old tool-call history to free space
- Key challenge: agentic trajectories are expensive — each rollout involves real environment interaction (web browsing, code execution), not just token generation
- Practical finding: test-time compute scaling improves agent performance, but the gains are smaller and less consistent than for pure reasoning

18

Part 5: Infrastructure for RL at Scale

19

The Bottleneck: Rollout Generation

- In RL for LLMs, the training loop has two phases: **generation** (produce rollouts) and **training** (compute gradients)
- **Generation dominates**: producing $G = 64$ responses per prompt with a 70B model is far more expensive than the GRPO gradient update
- Naive approach: generate → train → generate → train (serial). Training GPUs idle during generation, and vice versa.
- **Generation : training compute ratio** ≈ 20:1 to 100:1 — the gradient step is cheap relative to rollout production
- For agentic tasks, the problem is even worse: each rollout requires real environment interaction (code sandbox, web browser, API calls) in addition to model inference

20

The Actor–Verifier–Learner Architecture

- Modern systems separate training into **specialized components running asynchronously**:
- **Actors**: generate trajectories using inference-optimized serving (vLLM, SGLang, tensor parallelism)
- **Environments / tools**: execute actions and return observations (sandboxed containers, web browsers, APIs)
- **Verifiers**: run tests, judges, constraint checks, and reward assembly
- **Learners**: consume completed trajectories and perform GRPO/PPO gradient updates
- **Task samplers**: decide what tasks to generate next — focus on frontier difficulty (mix of success/failure)
- When the learner finishes an update → push new checkpoint to actors → actors start generating with updated policy. **Slight staleness** (1–2 updates old) is handled gracefully by PPO clipping.
- Fully asynchronous architectures achieve **5× faster training** for agentic tasks (1 day vs. 5 days) and near-100% GPU utilization vs. ~50% with synchronous RL

21

Scale of Production RL

- **DeepSeek-R1**: GRPO on thousands of H800 GPUs, 100K+ problems, $G = 64$ rollouts per problem, thousands of RL steps
- **DeepSeek-V3.2**: post-training compute exceeds **10% of pre-training cost**. 1,800+ environments and 85,000+ agentic tasks across search, coding, and tool use. Specialist distillation + mixed RL. Gold-medal IMO and IOI.
- **GLM-5**: 100K Ascend 910C chips, async 'slime' architecture, three-stage sequential RL with cross-stage distillation
- **Qwen 3**: GRPO with progressive difficulty curriculum + on-policy distillation for reasoning
- **The compute budget for RL is now comparable to pre-training** — it's no longer a small fine-tuning step, it's a major training phase
- A single frontier RL run costs **millions of GPU-hours** — most of that is rollout generation

22

Part 6: Agentic RL — The Newest Frontier

23

From Reasoning to Agency

- Reasoning RL focuses on **single-turn problem solving** — the model thinks and produces an answer
- **Agentic RL** extends this to **multi-turn interaction with environments**: web browsing, code execution, terminal commands, API calls
- The model must: (1) observe environment state, (2) decide on an action (search, click, run command), (3) observe the result, (4) plan the next action
- Formally: agentic RL is a **POMDP** (partially observable MDP) over multi-turn trajectories, as opposed to the degenerate single-step MDP of standard LLM RL
- The policy is $\pi_\theta(a_t|h_t)$ where $h_t = (q, o_1, a_1, o_2, a_2, \ldots, o_t)$ is the full interaction history
- This is the fastest-moving and least mature part of the frontier RL stack

24

Agentic Reward Stacks

- Agentic RL rarely uses a single scalar reward — it builds a **reward stack** from multiple components:
- **Outcome reward**: did the task succeed? (binary or graded)
- **Process reward**: were intermediate steps useful? (valid tool syntax, correct document retrieved, progress toward goal)
- **Cost penalty**: tokens, latency, tool calls, retries (prevents runaway verbosity and tool abuse)
- **Safety penalty**: did the agent violate constraints or attempt unsafe actions?
- Typical composite: $R(\tau) = w_{\text{out}} r_{\text{out}} + \sum_t w_{\text{proc}} r_{\text{proc}}(h_t, a_t) - \lambda_{\text{cost}} C(\tau) - \lambda_{\text{safe}} S(\tau)$
- **Principle**: use exact verifiers wherever possible, process rewards only when reliable, heuristic shaping only where necessary. Bad reward weights produce hacking immediately.

25

Key Differences from Reasoning RL

- **Longer episodes**: 50–200+ turns with environment interaction vs. a single response
- **Delayed, sparse reward**: did the web search eventually find the answer? Reward comes at the end of a long trajectory.
- **Environment cost**: each rollout requires running a real (or simulated) environment — much more expensive than text generation
- **Credit assignment**: which of 50 actions was critical? Token masking is essential — gradients should only flow through the agent's own decisions, not tool outputs
- **Action space mismatch**: policy operates at **token level** but reward is at **trajectory or tool-call level** — a central design challenge
- **PPO may outperform GRPO here**: long horizons and delayed reward make per-token credit assignment (via GAE + value model) more valuable than per-episode group normalization
- **DeepSeek-V3.2's approach**: generate 1,800+ synthetic environments with diverse tasks, run mixed RL with both reasoning and agentic rewards simultaneously

26

Agentic RL: Failure Modes and Guardrails

- **Reward hacking**: exploit the verifier instead of solving the real task — frontier reasoning models have been observed modifying test cases, accessing answer keys, and exploiting loopholes
- **Process-reward gaming**: optimize local heuristics without improving final success
- **Verbosity and retry loops**: generate more tokens/tool calls because reward doesn't penalize waste strongly enough
- **Unsafe tool behavior**: agent learns high-reward but prohibited action sequences
- **Distribution fragility**: agent performs well on known environments but breaks under small task variations
- **Guardrails**: hidden tests, multiple validators, diversity-promoting sampling, cost penalties, constrain dangerous actions explicitly, log entire trajectories and inspect failure clusters

Part 7: The Complete Picture

28

Interactive Demo: Full Pipeline Visualization

🚀 **Interactive Demo:** pipeline_demo.html

29

Frontier Model Recipes (2025–2026)

Model	Reasoning RL	Agentic Training	Distillation	Key Innovation
DeepSeek-R1	GRPO + verifiable rewards	Limited	Rejection sampling → smaller models	R1-Zero showed emergent CoT from pure RL
DeepSeek-V3.2	Mixed RL (GRPO), >10% of pre-train compute	1,800+ envs, 85K+ tasks, real tools	Specialist distillation into unified model	Single-stage mixed RL; thinking integrated into tool-use; IMO/IOI gold
GLM-5	GRPO → Agentic RL → General RL	Sequential stage after reasoning	Cross-stage distillation	Async 'slime' infra, 100K chips
Qwen 3.x	GRPO + progressive curriculum	Supported	On-policy distillation	Thinking/non-thinking mode switching
OpenAI GPT-5.x	RL with verifiable rewards	Native tool calling	Unknown	Pioneered test-time compute at scale

All use the same core recipe: policy optimization + verifiable rewards + multi-stage/mixed pipeline. They differ in scale, infrastructure, and whether stages are sequential or unified.

30

The Unified Framework: From Base Model to Frontier Reasoner

- **Pre-training**: language, knowledge, basic capabilities from next-token prediction on trillions of tokens
- **Optimization algorithms**: PPO (per-token GAE, 4 models), GRPO (group-relative, 3 models), DAPO (GRPO that works at scale). All are policy gradient + baseline + clipping.
- **Reward signals**: reward models (Bradley-Terry, GenRM), DPO (implicit), verifiable rewards (exact), process reward models (per-step). The specification language of post-training.
- **Multi-stage pipeline**: SFT → reasoning RL → distillation → alignment. Each stage targets specific capabilities.
- **Distillation**: rejection sampling, specialist distillation, on-policy distillation. RL discovers capabilities, distillation makes them stable and efficient.
- **Test-time compute**: best-of-N, long CoT, tree search + PRM, budget forcing. Trade inference compute for accuracy.
- **Agentic RL** (this lecture): multi-turn interaction with environments, reward stacks, actor–verifier–learner architecture. The newest frontier.
- These components compose into the system that produces every frontier reasoning model.

31

What's Still Open

- **Generalizing RLVR**: current reasoning RL works best for math/code. Extending to domains where correctness is ambiguous — creative writing, nuanced advice, scientific reasoning without ground truth.
- **Credit assignment in long horizons**: agentic RL with 200-step episodes still struggles. Better process rewards and hierarchical RL are active areas.
- **Reward hacking in general alignment**: theoretically unavoidable. Mixing verified and preference rewards helps but doesn't eliminate it.
- **Compute efficiency**: RL is expensive. On-policy distillation, smarter sampling (dynamic sampling from DAPO), better inference engines, and smaller proxy models are all active fronts.
- **Unified vs. sequential training**: DeepSeek-V3.2 shows mixed RL can work; others still use sequential stages. Which is better at scale remains unsettled.
- **The frontier is moving fast** — the core ideas (policy optimization, reward specification, multi-stage training, test-time compute) will persist; specific methods will continue to evolve.

32

Summary

33

All Interactive Demos

🚀 **Interactive Demo:** pipeline_demo.html

34

Summary

- **Multi-stage pipeline**: SFT → reasoning RL (GRPO/DAPO + verifiable rewards) → distillation → general alignment. Ordering matters. Emerging alternative: single-stage mixed RL (DeepSeek-V3.2).
- **DeepSeek-R1**: pure RL on base model → emergent CoT, self-verification, backtracking. Multi-stage pipeline makes it deployable. RL discovers capabilities; distillation stabilizes them.
- **Distillation**: rejection sampling, specialist distillation, on-policy distillation. 10–100× cheaper than RL for comparable reasoning. R1 distilled models beat RL-trained models of the same size.
- **Test-time compute**: best-of-N, long CoT, tree search + PRM, budget forcing. Models trained with RL can trade inference compute for accuracy. Extends to agentic tasks.
- **Infrastructure**: async actor–verifier–learner architecture. Rollout generation is the bottleneck. Production RL costs millions of GPU-hours. Post-training compute now exceeds 10% of pre-training.
- **Agentic RL**: multi-turn environment interaction, reward stacks (outcome + process + cost + safety), credit assignment over long horizons. The newest and fastest-moving frontier.

RL Arc: The Complete Map

- **Optimization**: How to update the policy safely. PPO, GRPO, DAPO. The *mechanics* of RL.
- **Reward Signals**: What to optimize toward. Reward models, DPO, verifiable rewards, PRMs. The *specification* of RL.
- **Pipelines & Scaling**: How to compose the pieces. Multi-stage training, distillation, test-time compute, agentic RL, infrastructure. The *engineering* of RL.
- Together: the complete shortest path from pre-trained base model to frontier reasoning system.

36

Supplementary Resources

🚀 **Interactive Demo:** handout.html

37